JARED LEMLER

Software Engineer • Pacifica, CA (SF Bay Area) • jared@jaredlemler.com • jaredlemler.com • github.com/JL037 • linkedin.com/in/jaredlemler

SUMMARY

Software engineer who builds and runs production systems end to end, along with the tools that keep them healthy. Built and run VoxPort, a live federated podcast platform with LLM-driven incident response, and nfl-dagster, a Dagster + Snowflake + dbt pipeline with an LLM question tool that only states numbers it got from a query. Eight years as a journeyman electrician troubleshooting live systems built the habit of finding the cause, fixing it, and shipping the fix.

EXPERIENCE

VoxPort | Founder & Lead Engineer (voxport.live) | Oct 2025 to Present

TypeScript, Fastify, Next.js, Prisma Postgres, AT Protocol, Cloudflare R2, Fly.io, Vercel, Axiom

- **Built and operate a production platform end to end, solo:** creator studio, public listener pages, Bluesky OAuth identity, R2 media pipeline, browser-native recording booth (off-main-thread MP3 encoding, IndexedDB crash recovery), and Fly.io/Vercel deploys. Serves 13 creators and 489 published episodes, with ~12K RSS feed pulls/month from Spotify, Apple, and Pocket Casts.
- **Used an LLM agent to investigate production incidents:** ran it over Axiom logs and Postgres stats (pg_stat_statements, pg_stat_user_tables) to produce a root-cause audit of runaway database spend. It traced the cost to a firehose reconnect storm (~820 reconnects/day), one cursor row taking ~5k writes/day, and scheduled jobs re-running on every restart.
- **Cut production DB operations 96%** (~209k to ~3-8k/day) based on that audit by adding an in-memory creator-ID cache, lengthening the cursor flush interval, and batching writes into transactions, which removed ~60k table scans/day.
- **Built an event-driven ingestion service** on a 2,000+ events/sec Bluesky firehose. Server-side filtering across 7 collection types drops ~96% of events before they reach application code or the database.
- **Resolved an OAuth sign-in outage:** isolated Vercel IP routing as the root cause with cross-network probes, then routed client metadata through a Cloudflare proxy, restoring PDS sign-in from complete failure.
- **Automated incident response with an AI Slack bot:** a TypeScript service on GitHub Actions checks Axiom health metrics hourly and posts a Claude-written incident summary to Slack. A stall check catches silent failures that never log an error.
- **Automated sprint planning, issue tracking, and test generation** by connecting agents to Linear and Notion over MCP, using open-weights models.
- **Hardened untrusted input paths:** protected RSS import against SSRF and malicious uploads with RFC-1918 blocking, magic-number validation, and ffmpeg metadata stripping.

Talk | Backend Developer, Contract (Social Connection Platform) | Jan 2025 to Feb 2026

TypeScript, React, Vite, Tailwind CSS, Python, FastAPI, Cloudflare, Plausible Analytics

- Built internal analytics dashboards and automated reporting pipelines that turn raw event metrics into business insights, cutting manual reporting effort ~90%.
- Shipped a privacy-compliant web platform (React/Vite/Tailwind, FastAPI backend) serving 200+ active university students.
- Moved fast with the founder: turned Lovable prototypes into production features, like a QR-code trigger linking live events to onboarding.

Journeyman Electrician | Gaylor Electric • Wolverine Electric • IBEW | 2017 to 2025

- Led crews of 2-4 electricians on commercial jobsites, building and inspecting critical infrastructure to blueprint, schedule, and strict code where failure is not an option. Journeyman through ABC Indiana; IBEW member.

FEATURED PROJECT

nfl-dagster | NFL Contract Value Pipeline & Ask Tool (github.com/JL037/nfl-dagster • live demo)

Python, Dagster+ Serverless, Snowflake, dbt, Postgres, FastAPI, HTMX, Claude API, GitHub Actions, Railway

- **Built a production ELT pipeline:** Dagster orchestrates nflverse play-by-play, schedules, and OverTheCap contract data (2015 to present) into Snowflake RAW, STAGING, and MARTS layers via dbt; 20 assets with season partitions, refreshed on schedule by Dagster+ Serverless with no dependency on a local machine.
- **Stopped bad data before publish:** Each load deletes, copies, and audits inside one Snowflake transaction and rolls back on failure. Wrote a public postmortem on a race between blocking checks and downstream automation, and fixed it by validating before commit.
- **Shipped a serving layer:** An audited Dagster asset publishes finished marts to Postgres in a single-transaction swap, verified against Snowflake by row counts and primary-key fingerprints, so the website never queries the warehouse.
- **Modeled spend vs. performance:** QB cap share against EPA per dropback, with "worth it" defined as EPA above a fitted value line; documented edge cases (mid-season trades, missing cap rows) as tested decisions, 43/43 dbt tests passing in prod.
- **Grounded LLM question answering:** A Claude Haiku tool loop over select-only, parameterized, read-only queries with timeouts and row caps; code rejects any number not returned by a query, and out-of-scope questions get a "no data" reply. 81-question eval suite scores 78/81 (30/30 refusals) at ~$0.0005 and 4 s per question; chose the model over GLM-5.3 Flash on eval accuracy, cost, and latency.
- **Ran it like production:** Live public deploy with rate limits, spend caps, and no stored user data; required CI checks (pytest with mutation-checked tests, dbt parse, gitleaks full-history secret scan); Snowflake key-pair auth, least-privilege role, and a 10-credit monthly resource monitor.

ADDITIONAL PROJECTS

- **doc-lok** (github.com/JL037/doc-lok, on npm): agents kept spending tokens re-reading the same remote docs, so I wrote a zero-dependency TypeScript library and CLI that condenses Markdown links into compact markers and uses a two-phase HTTP HEAD/ETag cache to skip redundant downloads.
- **atproto-pro** (skills.sh): a published AT Protocol skill that gives AI coding agents working knowledge of the protocol. I also write about building on ATProto and RSS on pckt.blog.

SKILLS & TRAINING

Languages: TypeScript, JavaScript, SQL, Python

AI & Automation: LLM APIs (Claude), AI agents, LLM evals, Slack API bots, MCP integrations (Linear, Notion), AI coding tools (Claude Code, Devin CLI, OpenCode, Zed), context engineering, open-weights models

Data Engineering: Dagster (Dagster+ Serverless, partitions, asset checks, automation), Snowflake, dbt, write-audit-publish, ELT

Backend & Data: Node.js, Fastify, Next.js, React, FastAPI, Prisma, PostgreSQL (query and index tuning, pg_stat analysis), REST, WebSockets

Infra & Ops: Docker, Fly.io, Vercel, Railway, Cloudflare (R2, proxy), AWS (EC2, S3), Git/GitHub Actions, gitleaks, Axiom, Sentry, OpenTelemetry

Training: Self-taught software engineer with 1:1 mentorship from senior engineers (formerly at Facebook, Microsoft, and Prisma).